Assignment 5: Process Concept and the Process Control Block

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Deadline: two weeks from the date of issue — there is no automatic late penalty; if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).
Answer every question. Show all working for Numerical and Design questions — a bare final answer without the derivation earns partial credit only.
Reading: Silberschatz et al., *Operating System Concepts*, Ch. 3 §3.1, pp. 101–105 [2]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 3 §3.1–3.3, pp. 108–126 [3]; Bovet & Cesati, *Understanding the Linux Kernel*, Ch. 3, pp. 79–130 [1].

5.1 Conceptual

Q1. A classmate claims: "A process is just the executable file; if I run `./editor` three times there is one process with three windows, and all three share one PCB." Respond precisely [2].

(a) Distinguish a program from a process along four dimensions: what it is, passive vs. active, how many exist per run, and lifetime.

(b) For three runs of `./editor` from three terminals, state how many processes and how many PCBs exist, and which address-space region(s) the three may share.

(c) Explain why the number of windows on screen does not decide the number of processes, and state what you would ask the OS to settle the question.

Q2. The five-state model [3].

(a) Name the event that causes each of the six legal transitions.

(b) Explain why there is no waiting → running arrow and no ready → waiting arrow.

(c) Explain why Stallings' two-state model (running / not running) is too coarse, and what goes wrong if the dispatcher picks a not-running process that is actually blocked.

(d) Explain why the PCB must live in kernel memory rather than in the process's own heap. Give two distinct reasons.

Concepts That Click

5.2 Numerical

Q3. A single-CPU system has seven processes. At time t_0: `P1` is running; `P2`, `P3`, `P4` are ready (queue order `P2`, `P3`, `P4`); `P5` and `P6` are waiting for the disk; `P7` is new. The scheduler always dispatches the head of the ready queue, and processes join the ready queue at the tail. The following events occur in order:

(i) `P7` is admitted.

(ii) `P1`'s time slice expires; one dispatch follows.

(iii) The running process calls a blocking `read` on the keyboard; one dispatch follows.

(iv) `P5`'s disk I/O completes.

(v) The running process calls `exit`; one dispatch follows.

(a) After each event, give the running process, the ready queue (in order), and the waiting set.

(b) Count the total number of state transitions across all five events.

(c) Verify that the number of non-terminated processes is conserved except at the one event that changes it.

Q4. PCB arithmetic. A kernel's PCB contains: state (4 bytes), PID (4 bytes), program counter (8 bytes), 16 general-purpose registers of 8 bytes each, a flags register (8 bytes), a stack pointer (8 bytes), a page-table pointer (8 bytes), a priority (4 bytes), accounting fields totalling 24 bytes, and a pointer to an open-file table (8 bytes).

(a) Compute the size of one PCB in bytes.

(b) Which of these bytes form the processor-state information (Stallings) that must be saved when the process is interrupted? Compute that byte count.

(c) A server runs 3000 processes. How many KiB of kernel memory do the PCBs occupy (1 KiB = 1024 bytes)? Give the answer to two decimal places.

(d) Assign every field to one of Silberschatz's seven PCB categories.

5.3 Design

Q5. Write a short C program `logger.c` with: one global `int lines`, a function `void put(const char *s)` with a local `int n`, and a `main` that allocates a 256-byte buffer with `malloc` and calls `put` in a loop.

(a) For every named object (including the pointer to the buffer, the buffer itself, the parameter `s`, and the return address of `put`), state the region (text, data, heap, stack).

(b) `./logger` is started twice as `P1` and `P2`. Build a two-column table, in the style of the lecture's `./counter` example, of the PCB and address-space fields that differ and those that are shared.

(c) While `P1` is inside `put`, a timer interrupt hands the CPU to `P2`. List, in order, what the kernel saves into which PCB and what it loads, and explain why `P1` later resumes inside `put` with the correct value of `n`.

Concepts That Click

Q6. Linux maps the textbook states onto `task_struct`'s `state` field [1].

(a) Linux uses `TASK_RUNNING` for both ready and running. Design the extra information the kernel needs to tell the two apart, and say where it would naturally live.

(b) A process sleeping in `TASK_INTERRUPTIBLE` and one in `TASK_UNINTERRUPTIBLE` both correspond to which textbook state?

(c) Redraw the five-state diagram with each textbook state relabelled by the Linux flag(s) it corresponds to, and mark which textbook transitions become "no change of flag" in Linux.

References

[1] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3rd edition, 2006.

[2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

[3] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, 2001.

Concepts That Click